Student Academic Performance Prediction System

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460093	24103002874	BWU/BCA/24/097	ARMAN KUMAR

Of in partial fulfillment for the award of the degree

BACHELOR OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Management and Faculty of **Brainware University** for providing us with the opportunity, guidance and resources required to successfully complete this project. Their continuous support and encouragement have contributed to our learning experience.

We extend our sincere thanks to our project supervisor, **DR. RAHUL KUMAR GHOSH**, for valuable guidance, suggestions and feedback throughout the development of this project. Their guidance helped us understand the practical implementation of machine learning concepts and overcome technical challenges.

We are also thankful to the **Head of the Department** and all faculty members of the Department of Computational Science for providing a supportive academic environment and encouraging us during the project.

We would also like to acknowledge the developers and maintainers of Python, Pandas, NumPy, Scikit-learn and Streamlit, whose technologies were used in the development of this system.

Finally, we express our gratitude to our families and friends for their constant support and motivation throughout the project.

SIGNATURE: _____
PROJECT SUPERVISOR

ABSTRACT

The Student Academic Performance Prediction System is a machine learning-based application developed using Python to estimate students' final academic scores based on their academic performance and engagement-related information. The main objective of this project is to demonstrate the practical implementation of data preprocessing, supervised machine learning algorithms and predictive analysis in the educational domain.

The system uses six input features: Attendance Percentage, Internal Marks, Assignment Score, Previous Semester Score, Study Hours Per Day and Engagement Level. The target variable is Final_Academic_Score. The original dataset contains 5,010 records and 10 columns. After data cleaning, 4,980 records remain.

The system performs data cleaning by removing duplicate records and handling missing and invalid values. Numerical features are processed using median imputation and StandardScaler, while the categorical Engagement_Level feature is handled using most-frequent imputation and OneHotEncoder.

Three supervised regression algorithms—Linear Regression, Random Forest Regressor and Gradient Boosting Regressor—are trained and compared using an 80:20 train-test split. Linear Regression achieved the best result among the tested models, with a Mean Absolute Error (MAE) of 4.153748, Root Mean Squared Error (RMSE) of 5.239164 and R^2 score of 0.669052.

The selected model is integrated into a Streamlit web application. Users can enter student information and receive an estimated final academic score along with an application-defined performance category.

Overall, the project demonstrates the practical use of Python, data preprocessing, machine learning model comparison and web application development for student academic score prediction.

INDEX

❖ **ACKNOWLEDGEMENT**

❖ **ABSTRACT**

❖ **LIST OF FIGURE**

❖ **LIST OF TABLE**

❖ **LIST OF SCREENSHOTS**

1. **INTRODUCTION**
2. **LITERATURE SURVEY**
3. **SYSTEM ANALYSIS AND DESIGN**
4. **IMPLEMENTATION**
5. **OUTPUT SCREENSHOT**
6. **SOURCE CODE**
7. **TESTING AND DEBUGGING**
8. **CONCLUSION AND FUTURE ENHANCEMENT**
9. **REFERENCES**

1. INTRODUCTION

In modern education, technology and data-driven methods are increasingly being used to understand and improve students' academic performance. Academic results are influenced by several factors, including attendance, internal assessment marks, assignment performance, previous semester results and study habits. Considering these factors together can provide a better understanding of a student's academic progress.

Traditional academic evaluation mainly depends on examination results and manual assessment. Although these methods are important, they may not always provide an early estimate of a student's expected final score. Analysing multiple academic factors manually can also become time-consuming.

The Student Academic Performance Prediction System is a machine learning application developed using Python to estimate a student's Final_Academic_Score using six selected input features. It uses historical student data to train regression models and identify patterns between academic indicators and final scores.

The system uses Pandas and NumPy for data handling, Scikit-learn for data preprocessing and machine learning, and Streamlit for the user interface. The complete process includes data cleaning, feature selection, model training, model evaluation and prediction.

Three regression algorithms are compared: Linear Regression, Random Forest Regressor and Gradient Boosting Regressor. Based on the evaluation results, Linear Regression is selected as the final model because it provides the best performance among the tested algorithms on the selected test dataset.

The application provides a simple interface where users can enter attendance percentage, internal marks, assignment score, previous semester score, study hours and engagement level. It then displays an estimated final academic score and a performance category.

The system is intended as an academic prediction and analysis tool. Its output is an estimate based on the available dataset and should not be considered a guaranteed examination result.

2. LITERATURE SURVEY

Educational Data Mining (EDM) and Learning Analytics are important areas of research that focus on analysing educational information to understand student learning patterns and academic performance. These areas use statistical methods and machine learning algorithms to identify relationships between different academic factors.

Traditional student performance analysis often depends on individual parameters such as attendance percentage or examination marks. However, academic performance may be influenced by multiple factors at the same time. Therefore, analysing several variables together can provide a more comprehensive view of student performance.

Machine learning regression algorithms are commonly used when the expected output is numerical. In academic score prediction, regression models can estimate a student's final score based on previous academic information and other relevant indicators.

Linear Regression is a basic supervised learning algorithm that estimates the relationship between independent variables and a numerical target. It is relatively simple to understand and interpret. Random Forest Regressor uses multiple decision trees to generate predictions, while Gradient Boosting Regressor builds an ensemble of learners sequentially to reduce prediction errors.

Data preprocessing is another important part of machine learning. Missing values, duplicate records and inconsistent data can affect the performance of a model. Techniques such as median imputation, feature scaling and categorical encoding help prepare the dataset for model training.

Based on these concepts, the Student Academic Performance Prediction System combines data cleaning, preprocessing, regression model comparison and a web-based prediction interface. The project uses these methods to estimate the final academic score from multiple student-related features.

3. SYSTEM ANALYSIS AND DESIGN

The System Analysis and Design phase focuses on understanding the requirements of the Student Academic Performance Prediction System and planning how the different components will work together.

3.1 System Analysis

System analysis identifies the need for a data-driven method to estimate academic performance using multiple student-related factors.

Existing System:

In a traditional academic evaluation system, student performance is generally reviewed through attendance records, internal marks, assignment scores and examination results. These factors may be analysed separately, and estimating a final academic score using all of them can require manual effort.

Problems in the Existing System:

- Manual Analysis: Reviewing multiple academic indicators manually can be time-consuming.
- Separate Evaluation: Individual academic factors may not provide a complete picture of student performance.
- Delayed Understanding: Students may not receive an estimated performance overview until examination results are available.
- Lack of Combined Analysis: It may be difficult to evaluate attendance, study hours, previous results and engagement together.

Proposed System:

The proposed Student Academic Performance Prediction System uses machine learning algorithms to estimate a student's final academic score. It processes six academic and engagement-related input features and generates a numerical prediction through a trained regression model.

Advantages of the Proposed System:

- Combines six student-related input features.
- Automates the academic score estimation process.
- Compares multiple regression algorithms.
- Uses a consistent preprocessing pipeline.
- Provides predictions through a simple Streamlit web interface.
- Displays an estimated score with a performance category.

3.2 System Design

System design describes the structure of the application and explains how data moves through different stages of the machine learning process.

Modules of the System:

1. **Data Loading Module:** Loads the original student dataset using Pandas.
2. **Data Cleaning Module:** Checks duplicate records, missing values and invalid numerical entries.
3. **Feature Selection Module:** Selects six relevant input features and the target variable Final_Academic_Score.
4. **Data Preprocessing Module:** Handles missing values, scales numerical features and encodes categorical data.
5. **Model Training Module:** Trains Linear Regression, Random Forest Regressor and Gradient Boosting Regressor.
6. **Model Evaluation Module**: Compares the trained models using MAE, RMSE and R^2.
7. **Prediction Application Module:** Uses Streamlit to accept student details and display the predicted academic score.

‘

The complete preprocessing and model pipeline is saved so that the same transformations used during training are also applied to new student inputs.

SYSTEM ARCHITECTURE AND TECHNOLOGIES

System Architecture:

The Student Academic Performance Prediction System follows a sequential machine learning architecture. The original dataset is first loaded and cleaned. The selected features are then passed through a preprocessing pipeline. After preprocessing, the dataset is divided into training and testing sets, and three regression algorithms are trained and evaluated.

The best-performing model is saved and integrated into the Streamlit application. When a user enters student details, the saved pipeline processes the inputs and generates the predicted final academic score.

Technologies Used:

- **Programming Language:** Python
- **Data Manipulation:** Pandas and NumPy
- **Machine Learning:** Scikit-learn
- **Data Preprocessing:** SimpleImputer, StandardScaler, OneHotEncoder and ColumnTransformer
- **Web Application:** Streamlit
- **Model Saving:** Pickle/joblib-based saved model pipeline
- **Data Visualization:** Matplotlib and project visualization scripts

Dataset Source:

https://github.com/Raghulkumar2210/student-academic-performance-forecasting

Dataset File: student_performance_challenge33.csv.xls

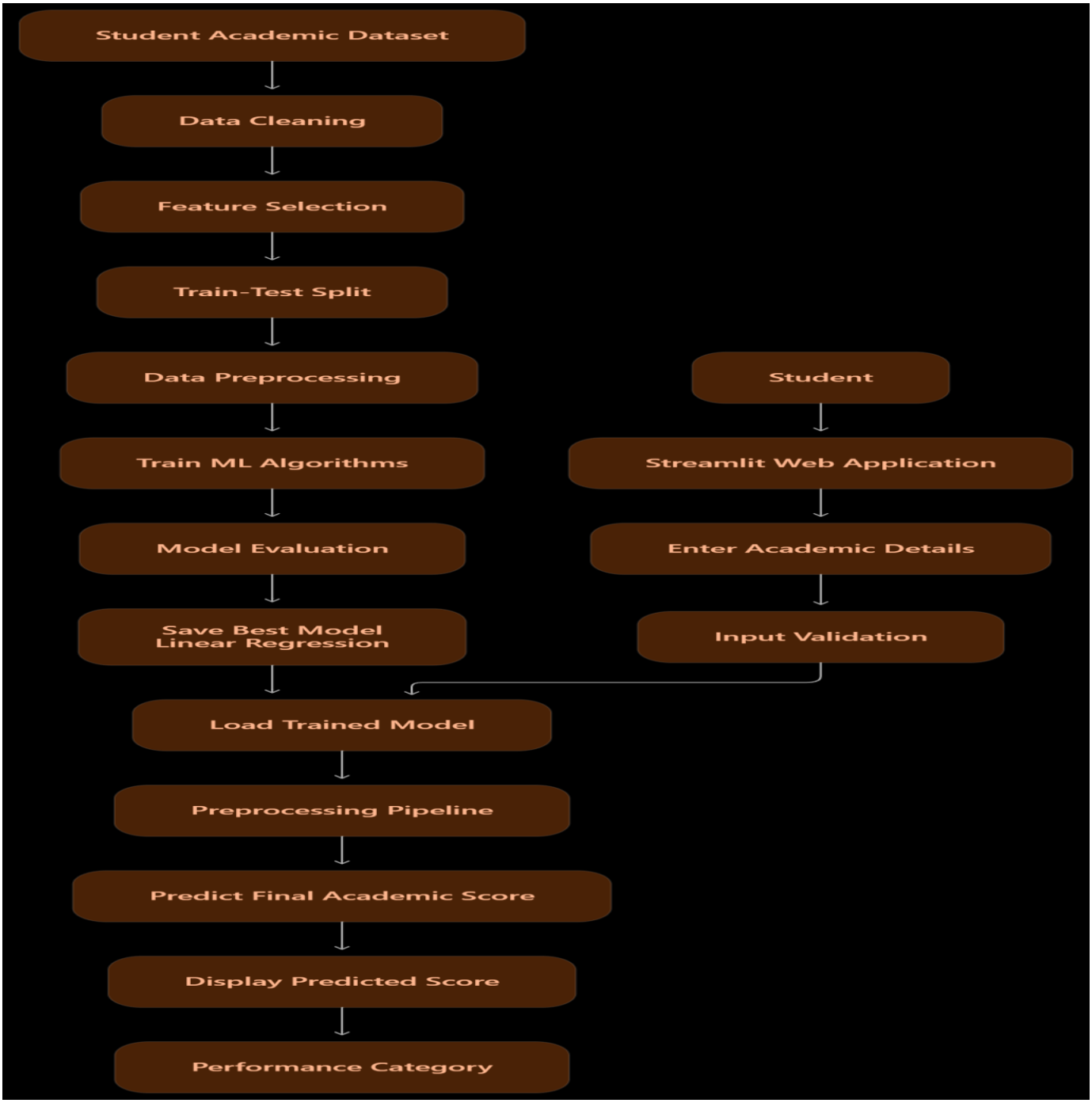


Figure 3.1 illustrates the complete workflow of the Student Academic Performance Prediction System, from dataset preparation and model training to student input and final score prediction.

COMPONENT ARCHITECTURE

The component architecture represents the relationship between the dataset, preprocessing pipeline, machine learning models and the Streamlit application.
The system consists of the following main components:

1. **Dataset:** Stores the original student academic records.
2. **Data Cleaning:** Removes duplicate records and handles missing or invalid values.
3. **Preprocessing Pipeline:** Performs numerical imputation, scaling and categorical encoding.
4. **Machine Learning Models:** Trains and compares three regression algorithms.
5. **Model Evaluation:** Calculates MAE, RMSE and R^2 to compare model performance.
6. **Saved Model:** Stores the selected Linear Regression pipeline.
7. **Streamlit Interface:** Accepts student inputs and displays the prediction.

The system design separates data preparation, model training and user interaction into different stages. This makes the application easier to understand, maintain and improve.

COMPONENT ARCHITECTURE DIAGRAM:

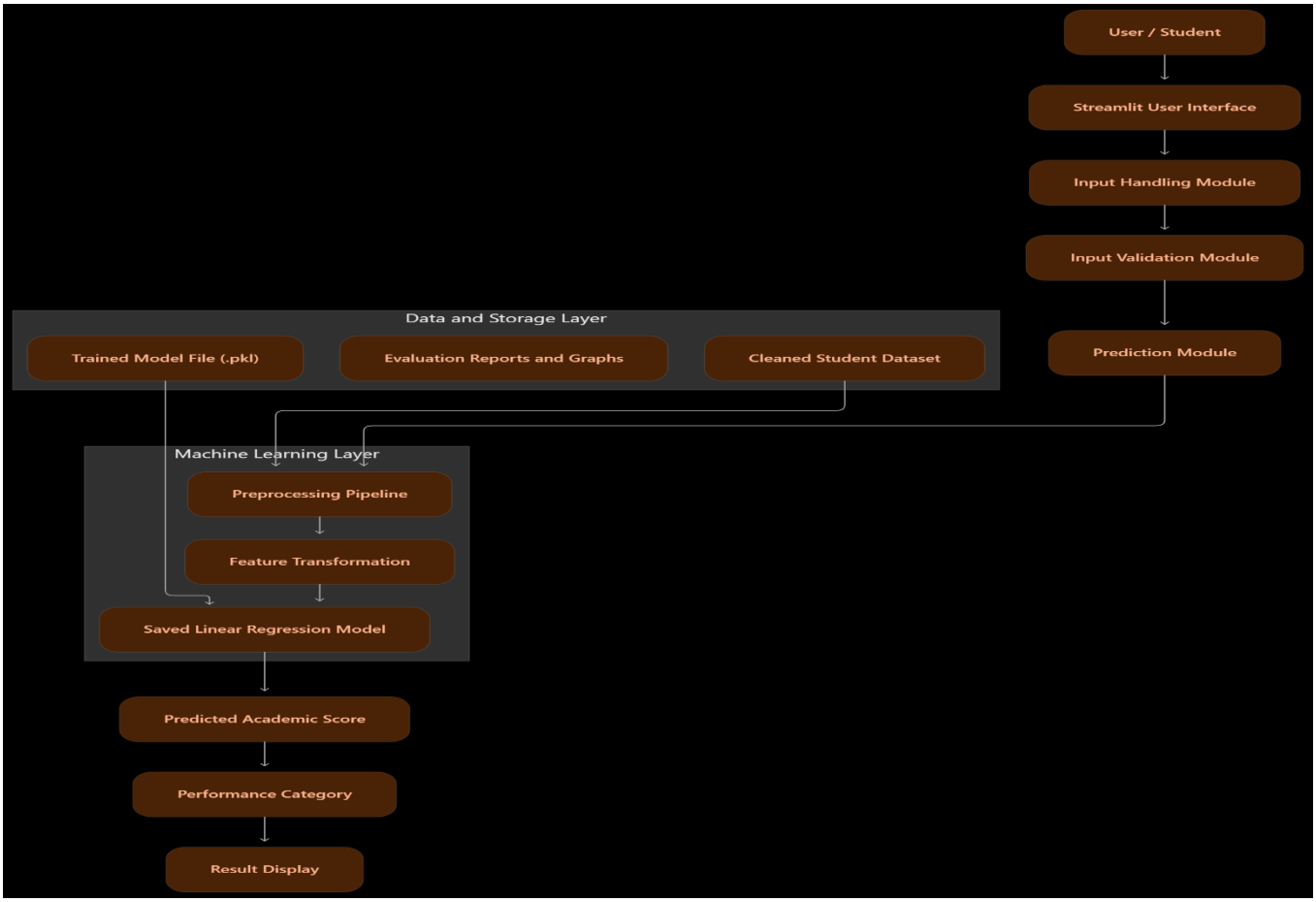


Figure 3.2 represents the component architecture of the proposed system. The Streamlit interface accepts academic details from students. The input handling and validation modules process the entered information. The machine learning layer uses the saved preprocessing pipeline and Linear Regression model to predict the final academic score. Finally, the system displays the predicted score and its performance category.

IMPLEMENTATION

The implementation phase involves converting the system design into a working machine learning application using Python. The project includes separate scripts for data analysis, data cleaning, model training, evaluation and feature analysis. A Streamlit application is used to provide the prediction interface.

Dataset Loading and Cleaning:

The dataset is loaded using the Pandas library. The original dataset contains 5,010 records and 10 columns. During the cleaning process, duplicate records, missing values and invalid numerical entries are checked.

Seven duplicate records are removed. Missing numerical values are handled using median imputation, while missing values in the Engagement_Level column are handled using the most frequent category.

After the cleaning process, the processed dataset contains 4,980 records and 10 columns. The cleaned dataset is saved as:

data/processed/student_performance_clean.csv

Data Preprocessing:

The project uses different preprocessing methods for numerical and categorical features.

Numerical features are processed using:

- SimpleImputer(strategy='median')
- StandardScaler()

The categorical Engagement_Level feature is processed using:

- SimpleImputer(strategy='most_frequent')
- OneHotEncoder(handle_unknown='ignore')

A ColumnTransformer combines these transformations. The preprocessing steps and machine learning estimator are joined using a Scikit-learn Pipeline.

This ensures that new inputs are processed using the same transformations applied during model training.

Train-Test Split:

The cleaned dataset is divided into training and testing sets using an 80:20 ratio. The train_test_split() function is used with test_size=0.2 and random_state=42.

- Total cleaned records: 4,980
- Training records: 3,984
- Testing records: 996

Machine Learning Algorithms:

Three regression algorithms are implemented and compared:

1. **Linear Regression:** Estimates a numerical output by modelling the linear relationship between input features and the target variable.
2. **Random Forest Regressor:** Uses multiple decision trees and combines their predictions to generate the final output. The project uses 200 estimators.
3. **Gradient Boosting Regressor:** Builds multiple learners sequentially, where later learners attempt to reduce the errors made by earlier learners.

Model Evaluation:

The models are evaluated using three performance metrics:

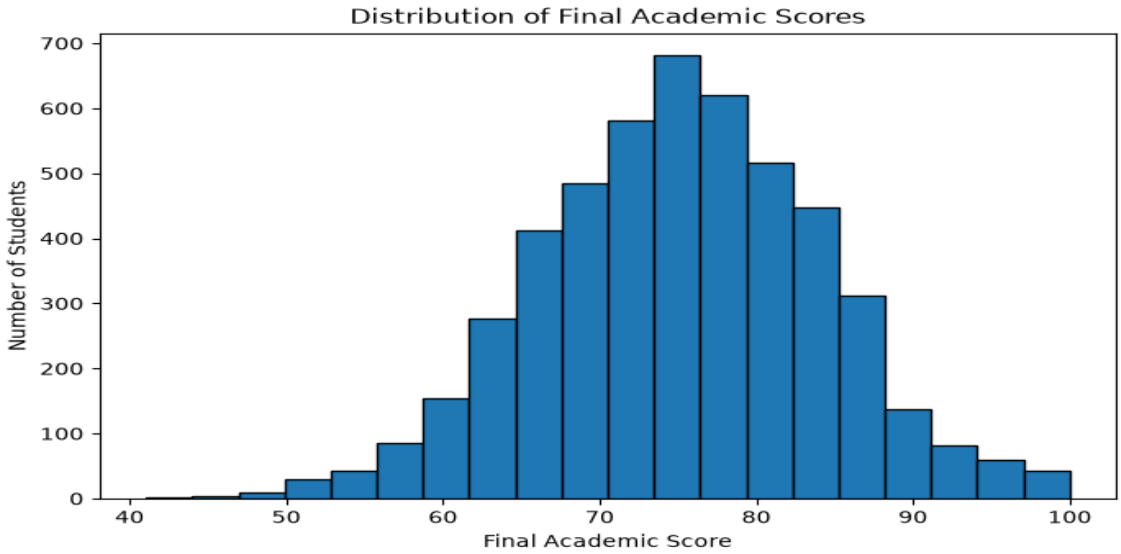
- **Mean Absolute Error (MAE):** Measures the average absolute difference between actual and predicted scores.
- **Root Mean Squared Error (RMSE):** Measures the square root of the average squared prediction error. Larger errors receive more weight.
- **R^2 Score:** Measures how much variation in the target variable is explained by the model compared with a baseline.

The model with the best evaluation results is selected for the final application.

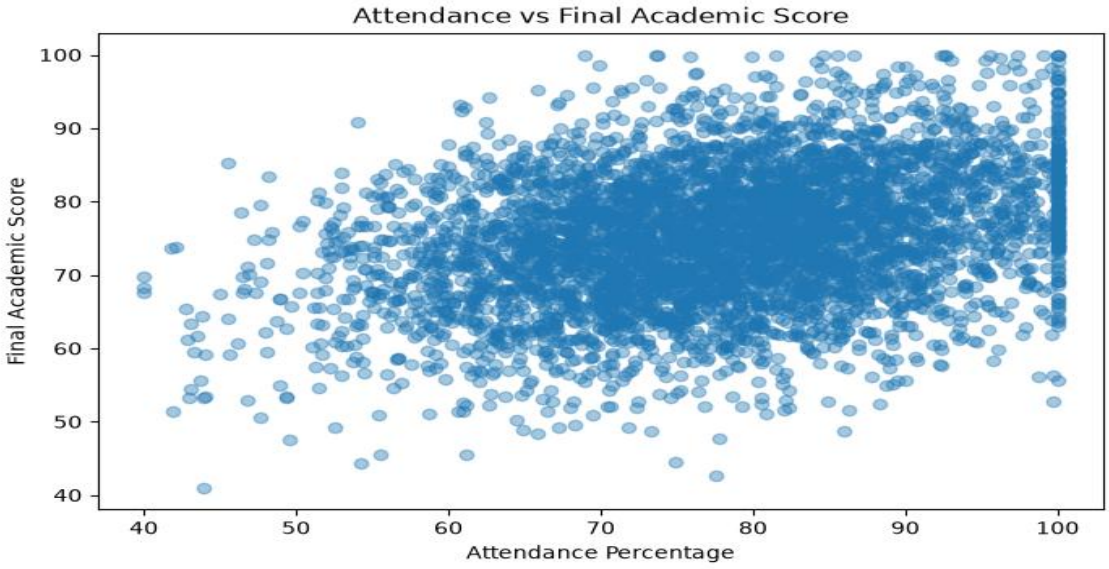
OUTPUT SCREENSHOT

The following figures represent the data visualizations generated during the analysis of the Student Academic Performance Prediction System. These visualizations help understand the distribution of academic scores and the relationship between different student-related features.

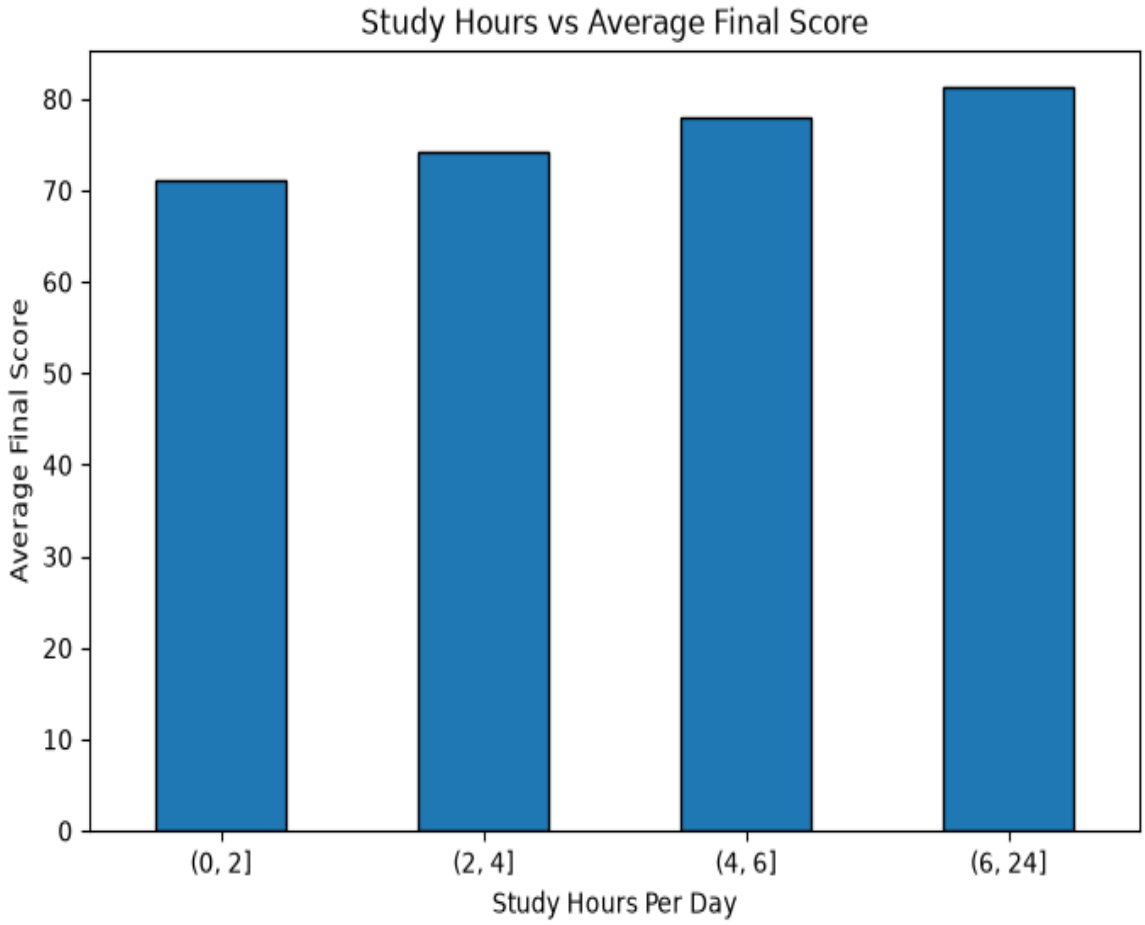
5.1 Final Academic Score Distribution



5.2 Attendance vs Final Academic Score

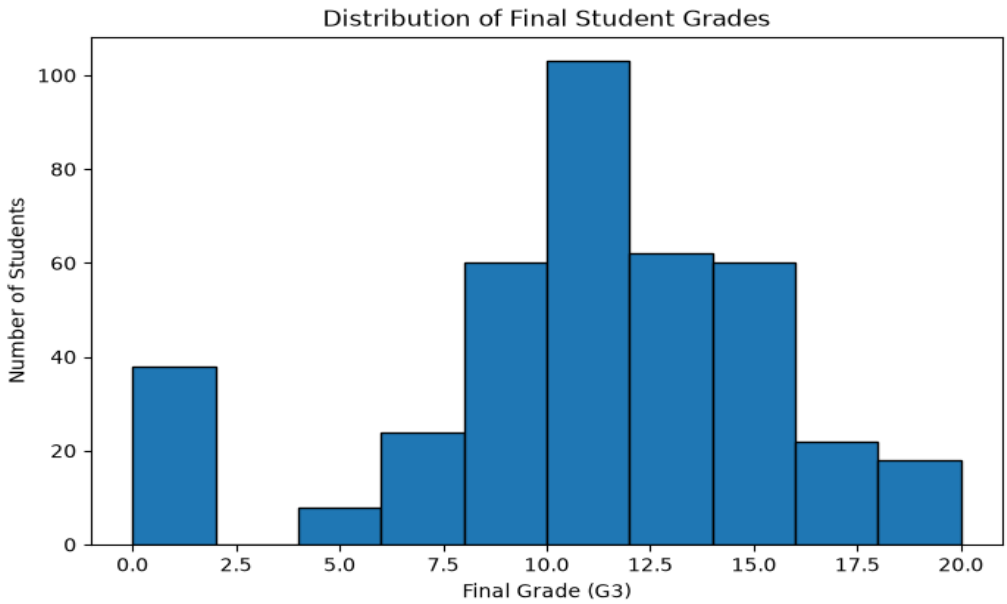


5.3 Study Hours vs Final Academic Score



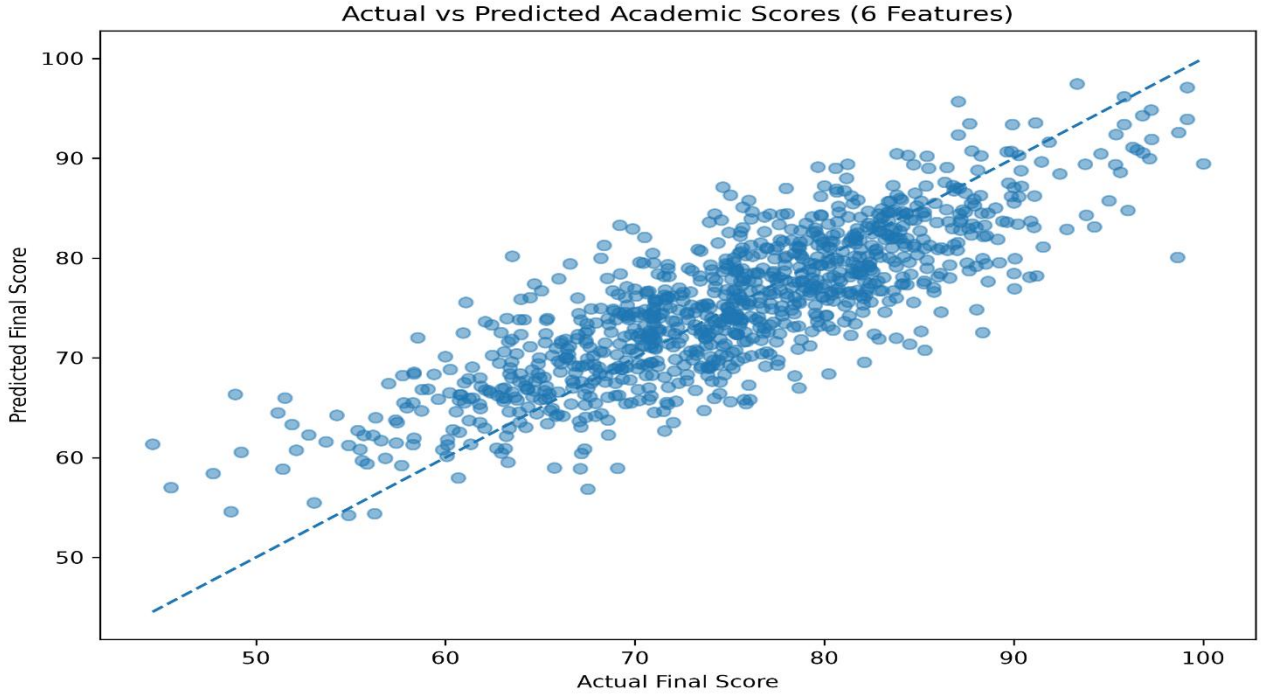
The graph represents the relationship between students' daily study hours and their final academic scores. It helps visually examine whether study hours show a pattern in relation to academic performance

5.4 Grade / Performance Distribution



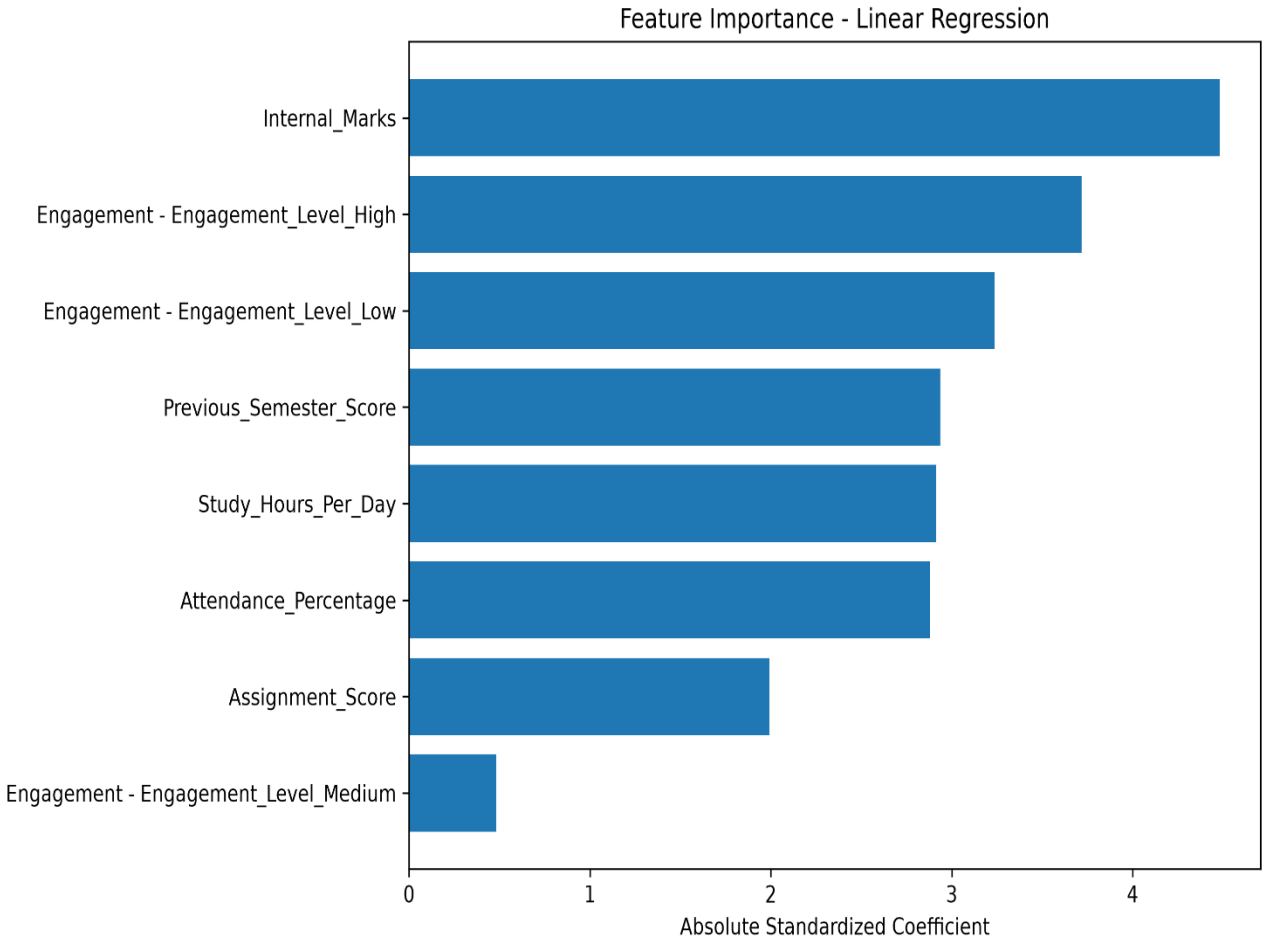
The graph represents the distribution of student performance categories available in the dataset or the corresponding project visualization. Insert the actual generated graph here.

5.5 Actual vs Predicted Regression Fit



The Actual vs Predicted graph compares the actual academic scores from the test dataset with the scores predicted by the selected Linear Regression model. The diagonal reference line represents the ideal situation where predicted values are equal to actual values.

5.6 Feature Analysis

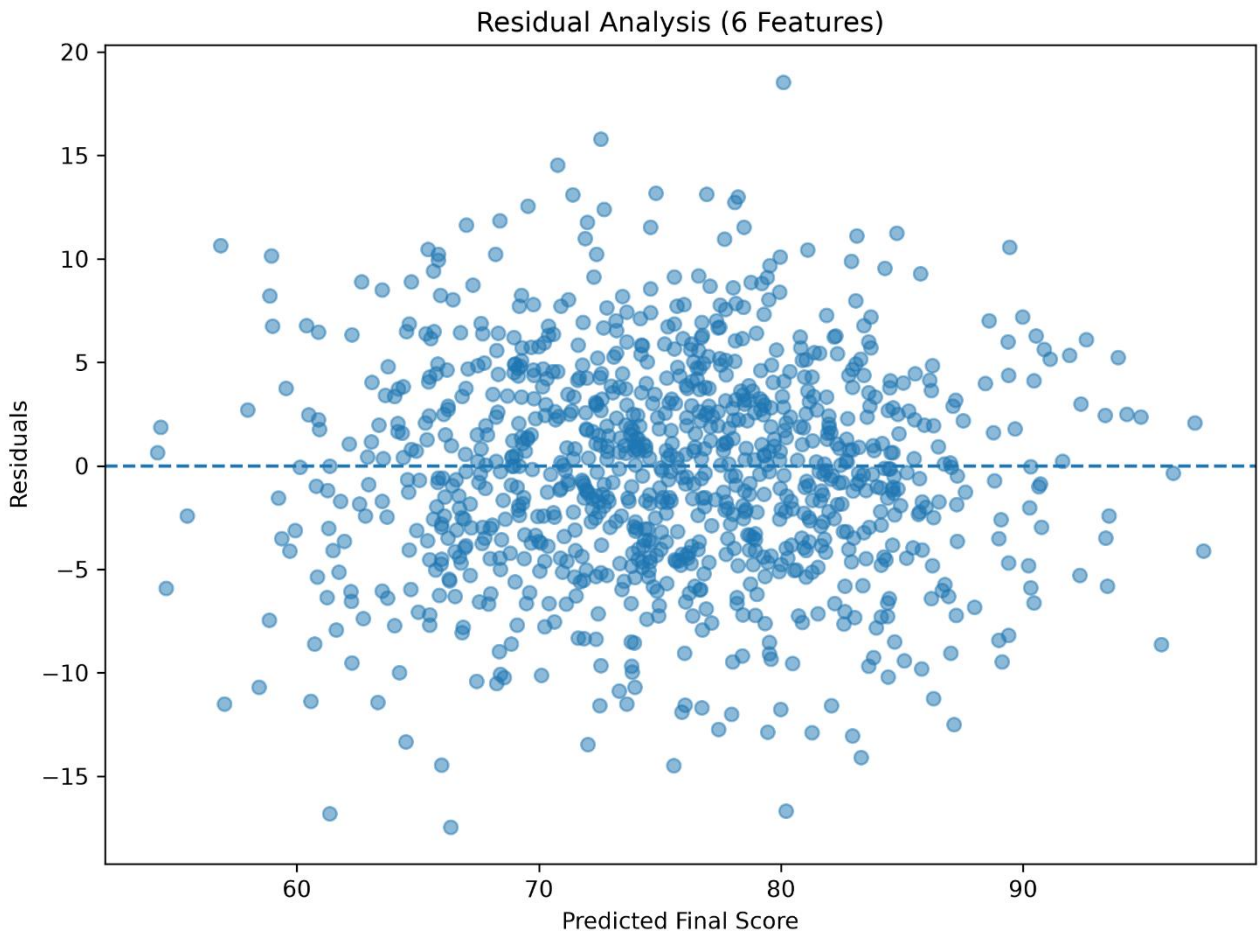


The project analyses the coefficients of the selected Linear Regression model to understand how the transformed input features contribute to its predictions.

The analysis includes academic features such as Internal_Marks, Previous_Semester_Score, Attendance_Percentage, Assignment_Score and Study_Hours_Per_Day, along with the encoded Engagement_Level categories.

These coefficients describe the fitted model and should not be interpreted as proof of cause-and-effect relationships.

5.7Residual Analysis



Residuals represent the difference between actual academic scores and predicted academic scores.

Residual = Actual Score − Predicted Score

The residual analysis graph helps examine the prediction errors made by the selected regression model. It provides a visual understanding of how far the predicted values differ from the actual values.

5.8Streamlit Interactive Prediction Output

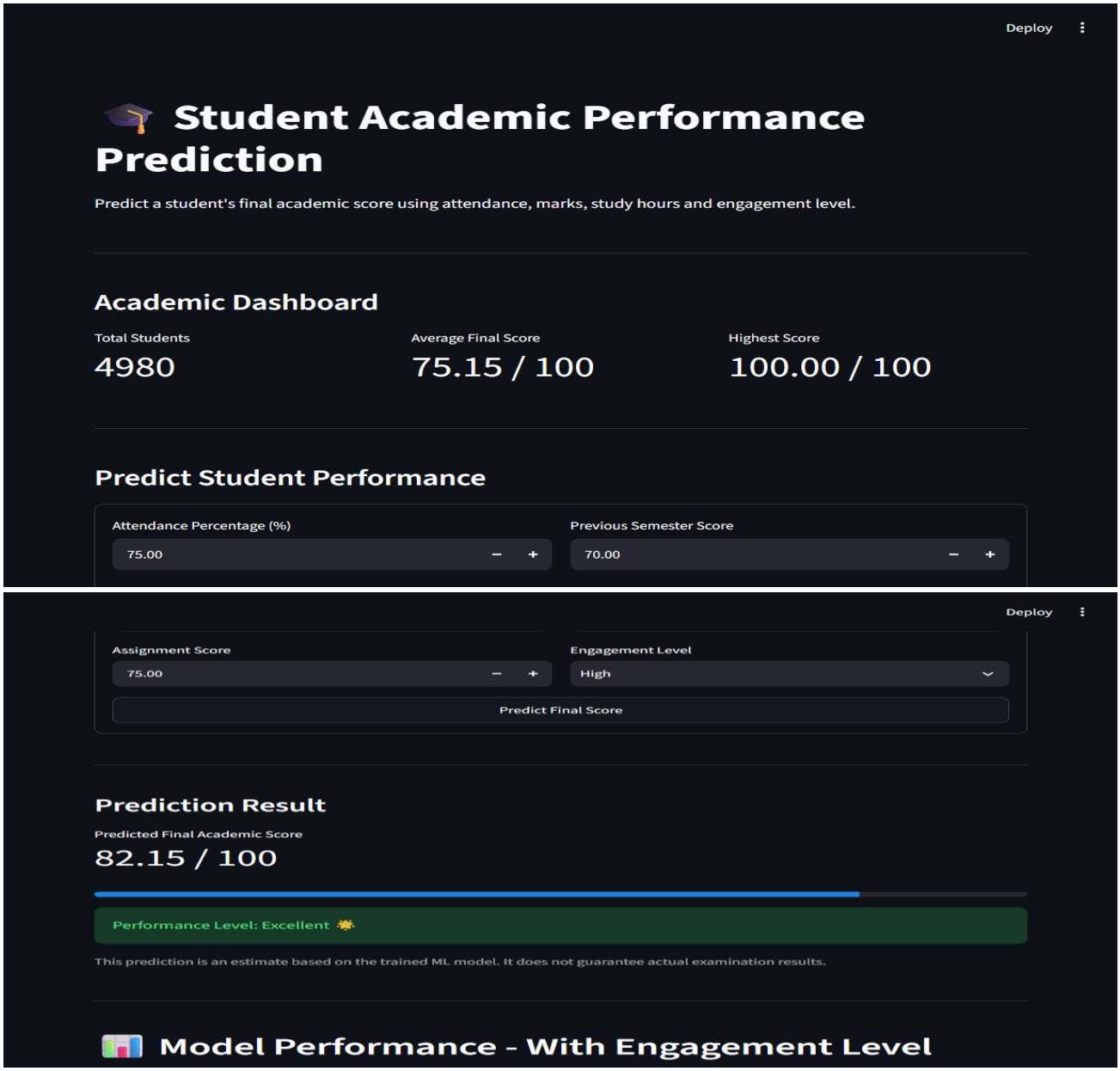


Fig. 5.8 – Student Academic Performance Prediction Application

The Streamlit application provides an interactive interface for entering student-related information. The user enters attendance percentage, internal marks, assignment score, previous semester score, study hours per day and engagement level.

After submitting the input values, the application loads the saved machine learning pipeline and generates an estimated Final_Academic_Score.

The predicted score and its corresponding performance category are displayed on the application screen.

SOURCE CODE

6.1 Project Structure

The Student Academic Performance Prediction System is organized into separate folders to maintain the dataset, trained models, project scripts and generated reports.

Main Project Files:

- **app.py –** Streamlit application used for student score prediction.
- **README.md –** Project overview and setup instructions.
- **requirements.txt** – Required Python libraries**.**
- **data/raw/** – Contains the original dataset**.**
- **data/processed/ –** Contains the cleaned dataset.
- **models/ –** Contains the saved machine learning models.
- **reports/ –** Contains model comparison results and generated figures.
- **src/ –** Contains Python scripts for data analysis, cleaning, model training, evaluation and feature analysis.

6.2 Data Cleaning Script

The data cleaning script is responsible for preparing the original dataset for machine learning. Its main operations include:

1. **Loading the dataset using Pandas.**
2. **Checking and removing duplicate records.**
3. **Identifying missing values.**
4. **Handling invalid numerical values.**
5. **Applying appropriate missing-value treatment.**
6. **Saving the cleaned dataset.**

```
import pandas as pd
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

# Load original dataset
input_path = BASE_DIR / "data" / "raw" / "student_performance_challenge33.csv.xls"

df = pd.read_csv(input_path)

print("Original Dataset Shape:", df.shape)

# Remove duplicate rows
df = df.drop_duplicates()

# Remove invalid values
df = df[
   (df["Attendance_Percentage"].between(0, 100)) &
   (df["Internal_Marks"].between(0, 100)) &
   (df["Assignment_Score"].between(0, 100)) &
```

```
    (df["Previous_Semester_Score"].between(0, 100)) &
    (df["Final_Academic_Score"].between(0, 100)) &
    (df["Study_Hours_Per_Day"].between(0, 24))
].copy()

# Handle missing values
df["Attendance_Percentage"] = df["Attendance_Percentage"].fillna(
    df["Attendance_Percentage"].median()
)

df["Assignment_Score"] = df["Assignment_Score"].fillna(
    df["Assignment_Score"].median()
)

df["Study_Hours_Per_Day"] = df["Study_Hours_Per_Day"].fillna(
    df["Study_Hours_Per_Day"].median()
)

df["Engagement_Level"] = df["Engagement_Level"].fillna(
    df["Engagement_Level"].mode()[0]
)

# Save cleaned dataset
output_path = BASE_DIR / "data" / "processed" / "student_performance_clean.csv"

df.to_csv(output_path, index=False)

print("\nCleaning Completed!")
print("Cleaned Dataset Shape:", df.shape)
print("Remaining Missing Values:", df.isnull().sum().sum())
print("Remaining Duplicates:", df.duplicated().sum())
print("Saved At:", output_path)
```

Model Training Script

The model training script is responsible for selecting the input features, defining the target variable, preparing the data and training the regression algorithms.

The six input features are:

- Attendance_Percentage
- Internal_Marks
- Assignment_Score
- Previous_Semester_Score
- Study_Hours_Per_Day
- Engagement_Level

The target variable is Final_Academic_Score.

The script creates a preprocessing pipeline using SimpleImputer, StandardScaler, OneHotEncoder and ColumnTransformer. It then trains and compares Linear Regression, Random Forest Regressor and Gradient Boosting Regressor.

```
import pandas as pd

import joblib


from pathlib import Path

from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer


from sklearn.linear_model import LinearRegression

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor


from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
BASE_DIR = Path(__file__).resolve().parent.parent

# Load cleaned dataset
df = pd.read_csv(
    BASE_DIR / "data" / "processed" / "student_performance_clean.csv"
)

# Five numerical features
numeric_features = [
    "Attendance_Percentage",
    "Internal_Marks",
    "Assignment_Score",
    "Previous_Semester_Score",
    "Study_Hours_Per_Day"
]

# Sixth categorical feature
categorical_features = [
    "Engagement_Level"
]

features = numeric_features + categorical_features

X = df[features]
y = df["Final_Academic_Score"]

# Split dataset: 80% training, 20% testing
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Records:", len(X_train))
print("Testing Records:", len(X_test))

# Preprocessing for numerical and categorical features
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_transformer, numeric_features),
    ("categorical", categorical_transformer, categorical_features)
])

# Define machine learning models
models = {
    "Linear Regression": LinearRegression(),

    "Random Forest": RandomForestRegressor(
```

```
        n_estimators=200,
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingRegressor(
        random_state=42
    )
}

results = []
trained_models = {}

# Train and evaluate models
for name, algorithm in models.items():

    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", algorithm)
    ])

    pipeline.fit(X_train, y_train)

    predictions = pipeline.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    rmse = mean_squared_error(y_test, predictions) ** 0.5
    r2 = r2_score(y_test, predictions)
```

```
    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2 Score": r2
    })

    trained_models[name] = pipeline

# Compare models
results_df = pd.DataFrame(results)
results_df = results_df.sort_values("RMSE")

print("\nMODEL COMPARISON - WITH ENGAGEMENT LEVEL")
print(results_df.to_string(index=False))
# Select model with lowest RMSE
best_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_name]
# Save new model separately
model_path = BASE_DIR / "models" / "best_model_academic_with_engagement.pkl"
joblib.dump(best_model, model_path)
# Save new comparison report
report_path = BASE_DIR / "reports" / "academic_model_comparison_with_engagement.csv"
results_df.to_csv(report_path, index=False)

print("\nBest Model:", best_name)
print("Model saved at:", model_path)
print("Comparison report saved at:", report_path)
```

6.4 Model Evaluation Script

The evaluation script checks the performance of the selected machine learning model using the testing dataset.

It calculates the following metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

The script also generates visualizations such as Actual vs Predicted and Residual Analysis graphs to understand the prediction performance.

```
import pandas as pd

import matplotlib.pyplot as plt

import joblib

from pathlib import Path

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

BASE_DIR = Path(__file__).resolve().parent.parent

# Load cleaned dataset

df = pd.read_csv(

    BASE_DIR / "data" / "processed" / "student_performance_clean.csv"

)

# Select all 6 features

features = [

    "Attendance_Percentage",

    "Internal_Marks",

   "Assignment_Score",

    "Previous_Semester_Score",

    "Study_Hours_Per_Day",

    "Engagement_Level"

]
```

```
X = df[features]
y = df["Final_Academic_Score"]
# Same train-test split used during training
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Load new trained model
model = joblib.load(
    BASE_DIR / "models" / "best_model_academic_with_engagement.pkl"
)
# Predict academic scores
predictions = model.predict(X_test)
# Evaluation metrics
mae = mean_absolute_error(y_test, predictions)
rmse = mean_squared_error(y_test, predictions) ** 0.5
r2 = r2_score(y_test, predictions)
print("\nMODEL EVALUATION RESULTS - 6 FEATURES")
print("-------------------------------------")
print("MAE:", round(mae, 6))
print("RMSE:", round(rmse, 6))
print("R2 Score:", round(r2, 6))
# Create figures folder if not present
figures_dir = BASE_DIR / "reports" / "figures"
figures_dir.mkdir(parents=True, exist_ok=True)
# Actual vs Predicted graph
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions, alpha=0.5)
plt.plot(
```

```
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)
plt.xlabel("Actual Final Score")
plt.ylabel("Predicted Final Score")
plt.title("Actual vs Predicted Academic Scores (6 Features)")
plt.tight_layout()
output_path = figures_dir / "actual_vs_predicted.png"
plt.savefig(output_path, dpi=300)
plt.close()
print("\nActual vs Predicted graph saved:", output_path)
# Residual Analysis graph
residuals = y_test - predictions
plt.figure(figsize=(8, 6))
plt.scatter(predictions, residuals, alpha=0.5)
plt.axhline(y=0, linestyle="--")
plt.xlabel("Predicted Final Score")
plt.ylabel("Residuals")
plt.title("Residual Analysis (6 Features)")
plt.tight_layout()
residual_path = figures_dir / "residual_analysis.png"
plt.savefig(residual_path, dpi=300)
plt.close()
print("Residual Analysis graph saved:", residual_path)
print("\nBoth graphs updated successfully!")
```

6.5 Feature Analysis Script

The feature analysis script examines the coefficients of the selected Linear Regression model

and generates a visualization of the transformed features.

```
import pandas as pd
import matplotlib.pyplot as plt
import joblib
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
# Load new six-feature model
model = joblib.load(
    BASE_DIR / "models" / "best_model_academic_with_engagement.pkl"
)
# Get preprocessing and trained Linear Regression model
preprocessor = model.named_steps["preprocessor"]
regressor = model.named_steps["model"]
# Get actual feature names after preprocessing
feature_names = preprocessor.get_feature_names_out()
# Get regression coefficients
coefficients = regressor.coef_
# Create feature importance dataframe
importance = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients,
    "Absolute Importance": abs(coefficients)
})
# Clean feature names for display
importance["Feature"] = (
    importance["Feature"]
    .str.replace("numeric__", "", regex=False)
    .str.replace("categorical__", "Engagement - ", regex=False)
```

```
)
importance = importance.sort_values(
    "Absolute Importance",
    ascending=False
)
print("\nFEATURE IMPORTANCE - WITH ENGAGEMENT LEVEL")
print(importance.to_string(index=False))
# Create graph
plt.figure(figsize=(10, 6))
plt.barh(
    importance["Feature"],
    importance["Absolute Importance"]
)
plt.xlabel("Absolute Standardized Coefficient")
plt.title("Feature Importance - Linear Regression")
plt.gca().invert_yaxis()
plt.tight_layout()
output_path = BASE_DIR / "reports" / "figures" / "feature_importance.png"
plt.savefig(output_path, dpi=300)
plt.close()
print("\nFeature importance graph saved successfully!")
print("Graph location:", output_path)
```

6.6 Streamlit Application

The Streamlit application provides the user interface for the Student Academic Performance Prediction System.

The application loads the saved model pipeline from:

models/best_model_academic_with_engagement.pkl

It accepts six input values from the user, creates a suitable input DataFrame and passes it to

the trained pipeline for prediction.

The application displays the estimated final academic score and assigns a performance category according to the defined score thresholds.

Performance Categories:

- Score 80 and above – Excellent
- Score 60 to below 80 – Good
- Score 40 to below 60 – Average
- Score below 40 – Needs Improvement

```
•
•  import streamlit as st
•  import pandas as pd
•  import joblib
•
•  from pathlib import Path
•
•  BASE_DIR = Path(__file__).resolve().parent
•
•  # Load new six-feature model
•  model = joblib.load(
•      BASE_DIR / "models" / "best_model_academic_with_engagement.pkl"
•  )
•
•  # Load dataset
•  df = pd.read_csv(
•      BASE_DIR / "data" / "processed" / "student_performance_clean.csv"
•  )
•
•  st.set_page_config(
•      page_title="Student Performance Prediction",
•      page_icon="🎓",
•      layout="wide"
•  )
•
•  # Header
•  st.title("🎓 Student Academic Performance Prediction")
•
•  st.write(
•      "Predict a student's final academic score using "
•      "attendance, marks, study hours and engagement level."
•  )
•
•  st.divider()
```

```
•
•   # Dashboard
•   st.subheader("Academic Dashboard")
•
•   col1, col2, col3 = st.columns(3)
•
•   col1.metric("Total Students", len(df))
•
•   col2.metric(
•       "Average Final Score",
•       f"{df['Final_Academic_Score'].mean():.2f} / 100"
•   )
•
•   col3.metric(
•       "Highest Score",
•       f"{df['Final_Academic_Score'].max():.2f} / 100"
•   )
•
•   st.divider()
•
•   # Prediction Form
•   st.subheader("Predict Student Performance")
•
•   with st.form("prediction_form"):
•
•       col1, col2 = st.columns(2)
•
•       with col1:
•           attendance = st.number_input(
•               "Attendance Percentage (%)",
•               min_value=0.0,
•               max_value=100.0,
•               value=75.0
•           )
•
•           internal_marks = st.number_input(
•               "Internal Marks",
•               min_value=0.0,
•               max_value=100.0,
•               value=70.0
•           )
•
•           assignment_score = st.number_input(
•               "Assignment Score",
•               min_value=0.0,
•               max_value=100.0,
```

```
            value=75.0
        )

    with col2:
        previous_score = st.number_input(
            "Previous Semester Score",
            min_value=0.0,
            max_value=100.0,
            value=70.0
        )

        study_hours = st.number_input(
            "Study Hours Per Day",
            min_value=0.0,
            max_value=24.0,
            value=4.0
        )

        engagement = st.selectbox(
            "Engagement Level",
            options=sorted(df["Engagement_Level"].dropna().unique()),
            index=0
        )

    submitted = st.form_submit_button(
        "Predict Final Score",
        use_container_width=True
    )

# Prediction Result
if submitted:

    input_data = pd.DataFrame([{
        "Attendance_Percentage": attendance,
        "Internal_Marks": internal_marks,
        "Assignment_Score": assignment_score,
        "Previous_Semester_Score": previous_score,
        "Study_Hours_Per_Day": study_hours,
        "Engagement_Level": engagement
    }])

    prediction = model.predict(input_data)[0]

    prediction = max(0, min(100, prediction))

    st.divider()
```

```
    st.subheader("Prediction Result")

    st.metric(
        "Predicted Final Academic Score",
        f"{prediction:.2f} / 100"
    )

    st.progress(prediction / 100)

    if prediction >= 80:
        st.success("Performance Level: Excellent 🌟")

    elif prediction >= 60:
        st.info("Performance Level: Good 👍")

    elif prediction >= 40:
        st.warning("Performance Level: Average 📄")

    else:
        st.error("Performance Level: Needs Improvement")

    st.caption(
        "This prediction is an estimate based on the trained ML model. "
        "It does not guarantee actual examination results."
    )

st.divider()

# Updated Model Performance
st.header("📊 Model Performance - With Engagement Level")

col1, col2, col3 = st.columns(3)

col1.metric("MAE", "4.15", delta="-0.52")
col2.metric("RMSE", "5.24", delta="-0.66")
col3.metric("R² Score", "0.6691", delta="+0.0893")

st.caption(
    "Selected Model: Linear Regression | Six input features"
)

st.subheader("Actual vs Predicted Scores")

actual_graph = BASE_DIR / "reports" / "figures" / "actual_vs_predicted.png"
```

```
st.image(str(actual_graph), use_container_width=True)

st.subheader("Residual Analysis")

residual_graph = BASE_DIR / "reports" / "figures" / "residual_analysis.png"

st.image(str(residual_graph), use_container_width=True)

st.divider()

st.caption(
    "Machine Learning Project | Student Academic Performance Prediction"
)

```

GitHub Repository:

https://github.com/armankumar12-git/Student-Academic-Performance-Prediction.git

TESTING AND DEBUGGING

Testing and debugging are important phases in the development of the Student Academic Performance Prediction System. Testing helps verify whether the different components of the application work as expected, while debugging helps identify and correct errors during development.

The system includes different stages such as dataset loading, data cleaning, feature preprocessing, model training, model evaluation and prediction through the Streamlit interface.

Types of Testing

1. Data Loading Testing

This testing checks whether the original dataset can be loaded successfully using Pandas and whether the required columns are available.

2. Data Cleaning Testing

This testing checks whether duplicate records are removed and missing or invalid values are handled by the cleaning process.

3. Preprocessing Testing

This testing checks whether numerical features are processed using median imputation and StandardScaler and whether the Engagement_Level feature is correctly encoded.

4. Model Testing

The trained regression algorithms are evaluated using the testing dataset. MAE, RMSE and R^2 are calculated to compare the model results.

5. Application Testing

The Streamlit application is checked to verify that the saved model loads, accepts valid student inputs and displays a predicted academic score.

Overall Testing Objective:

The objective of testing is to ensure that the data preparation, machine learning pipeline and prediction interface work together correctly. Actual test outcomes should be recorded after running the relevant tests.

TEST CASES

The following test cases describe the main functional checks relevant to the Student Academic Performance Prediction System.

Test ID	Description	Input	Expected Result	Actual Output	Status
TC-1	Dataset Loading	Run data cleaning script	Dataset loads successfully	To be filled	To verify
TC-2	Duplicate Removal	Original dataset	Duplicate records are removed	To be filled	To verify
TC-3	Missing Value Handling	Dataset containing missing values	Configured imputers handle missing values	To be filled	To verify
TC-4	Model Loading	Launch Streamlit application	Saved model pipeline loads	To be filled	To verify
TC-5	Valid Prediction	Enter six valid feature values	Predicted score and category are displayed	To be filled	To verify
TC-6	Engagement Input	Select an engagement category	Pipeline processes the selected category	To be filled	To verify

Debugging:

During development, debugging involves checking error messages, verifying feature names and data types, confirming the saved model path and ensuring that the application uses the correct trained model.

The actual output and status of each test should be updated after performing the test in the running application.

8. CONCLUSION AND FUTURE ENHANCEMENT

The Student Academic Performance Prediction System demonstrates the practical use of machine learning for estimating student academic scores using multiple academic and engagement-related factors.

The project includes dataset loading, data cleaning, feature selection, preprocessing, regression model training and performance evaluation. Three algorithms—Linear Regression, Random Forest Regressor and Gradient Boosting Regressor—were compared using an 80:20 train-test split.

Among the tested models, Linear Regression achieved the best evaluation results with an MAE of 4.153748, RMSE of 5.239164 and R^2 score of 0.669052. Therefore, it was selected for integration into the final application.

The system uses a Scikit-learn Pipeline to combine preprocessing and prediction. A Streamlit interface allows users to enter six student-related features and view the estimated final academic score along with an application-defined performance category.

Overall, this project provides practical experience in Python programming, data preprocessing, supervised machine learning, model evaluation and web application development. The prediction should be treated as an estimate based on the available dataset rather than a guaranteed academic result.

Future Enhancement

Although the current system performs academic score prediction, several improvements can be considered in future versions:

1. Larger Dataset: The system can be trained and evaluated using larger and more diverse student datasets.

2. Cross-Validation: Cross-validation can be used to evaluate model performance more reliably across different data splits.

3. Hyperparameter Tuning: Model parameters can be optimized to explore possible improvements in prediction performance.

4. Additional Machine Learning Algorithms

More regression algorithms can be tested and compared to identify models that may perform better on different student datasets.

5. Improved Feature Analysis

Permutation importance and other model interpretation techniques can be explored to understand the contribution of different input features more effectively.

6. Improved Web Interface

The Streamlit application can be enhanced with additional visualizations, improved input validation

and a more interactive dashboard.

7. Database Integration

A database may be added in a future version to store student information and prediction history securely.

8. Model Deployment

The application can be deployed on a suitable cloud platform so that it can be accessed through a web browser.

With these improvements, the Student Academic Performance Prediction System can be extended into a more comprehensive academic performance analysis application.

REFERENCES

Scikit-learn – Official Documentation

https://scikit-learn.org/stable/

(For regression algorithms, preprocessing techniques, model pipelines and evaluation metrics.)

Pandas – Official Documentation

https://pandas.pydata.org/docs/

(For dataset loading, DataFrame manipulation, duplicate removal and missing-value handling.)

NumPy – Official Documentation

https://numpy.org/doc/

(For numerical operations and array processing.)

Streamlit – Official Documentation

https://docs.streamlit.io/

(For developing the interactive web application and displaying model predictions.)

Python Software Foundation

https://docs.python.org/3/

(For Python programming language and standard library documentation.)

Student Academic Performance Forecasting Dataset

https://github.com/Raghulkumar2210/student-academic-performance-forecasting

(For the student academic dataset used in this project.)

Scikit-learn User Guide – Linear Models

https://scikit-learn.org/stable/modules/linear_model.html

(For understanding Linear Regression and related model concepts.

Class notes and study materials provided by Brainware University

(Deprtment of Computational Sciences, Bachelor of Computer Application curriculum)